PYTHON – DATA ANALYSIS

Vivek Kumar

From Owner of this Document

Great! You've successfully downloaded the interview tips and questions list.

If you have any doubts or need clarity, feel free to *book a 15-minute slot* with me.

https://topmate.io/vivek_kumar52/1381028

Looking for a confidence boost? You can also *schedule a mock interview session* at your convenience.

https://topmate.io/vivek_kumar52/965048

Let's make sure you're fully prepared!

Your trustworthy Mentor,
Vivek Kumar

Power BI Questions Document - https://topmate.io/vivek_kumar52/1374738

SQL Questions Document – https://topmate.io/vivek_kumar52/1617341

Excel document - https://topmate.io/vivek_kumar52/1643825

Dax Document - https://topmate.io/vivek_kumar52/1457572

CONTENTS

INTERVIEW PREPARATION

🎯 BEFORE THE INTERVIEW – PREPARATION TIPS

Tip	Why It Matters
✅ Master the Basics	Know Python fundamentals: data types, control flow, functions, OOP
✅ Practice Coding	Use platforms like LeetCode, HackerRank, and Codility
✅ Understand Real-World Scenarios	Practice file handling, data processing, APIs, exception handling
✅ Revise Key Libraries	Especially pandas, numpy, requests, os, json, datetime
✅ Build Projects	Small projects give confidence & are great for discussion during interviews
✅ Know Your Resume	Be ready to explain everything you've mentioned — especially projects and tools
✅ Mock Interviews	Practice with a mentor for realistic Q&A flow

💬 DURING THE INTERVIEW – PERFORMANCE TIPS

Tip	Why It Matters
🧠 Think Out Loud	Shows your thought process — especially in coding or logic questions
✍️ Use a Structured Approach	Break problems into steps: input, logic, output, edge cases
❓ Clarify the Question	Don't assume; ask questions to understand the requirement
🧹 Write Clean Code	Use meaningful names, modular code (functions), and avoid clutter
🛑 Don't Panic on Unknowns	Be honest. Say, "I haven't worked on this directly, but I'd approach it like..."
💡 Share Alternatives	If you know multiple solutions, share the trade-offs (e.g. loop vs. list comprehension)
🔁 Ask for Feedback	Especially in coding tasks — "Would you like me to optimize this?"

AFTER THE INTERVIEW – FOLLOW-UP TIPS

Tip	Why It Matters
Send a Thank-You Note	Shows professionalism and interest
Reflect on Questions	Note what went well and what needs improvement for next time
Fill the Gaps	If you missed something, study it right after the interview
Stay Consistent	Job hunting is a process — every interview helps you improve

BONUS TIPS SPECIFIC TO PYTHON INTERVIEWS

- Use **Pythonic solutions** where possible (e.g., list comprehensions, enumerate(), zip(), with open(), etc.)
- Be ready to write **code without an IDE**
- Know common **time and space complexity** of built-in operations
- Prepare for **unit testing basics** (unittest, pytest)
- Practice writing **clean and reusable functions**

PYTHON – AN OVERVIEW

Python is a high-level, interpreted programming language known for its simplicity, readability, and versatility. It was created by **Guido van Rossum** and first released in **1991**.

◇ KEY FEATURES OF PYTHON

- **Easy to Learn and Use** - Simple, readable syntax that closely resembles English.
- **Interpreted Language** - Python code is executed line by line, which makes debugging easier.
- **Dynamically Typed** - No need to declare variable types explicitly.
- **Object-Oriented & Functional** - Supports both object-oriented and functional programming paradigms.
- **Extensive Standard Library** - Comes with a rich set of modules and libraries for diverse tasks.
- **Cross-Platform** - Works on Windows, macOS, Linux, etc.
- **Large Community Support** - Active community with lots of open-source libraries and frameworks.

◇ POPULAR USES OF PYTHON

Area	Use Case Examples
Web Development	Django, Flask, FastAPI
Data Science	Pandas, NumPy, Matplotlib, Scikit-learn, Seaborn
Machine Learning	TensorFlow, PyTorch, XGBoost
Automation/Scripting	File handling, web scraping (Selenium, BeautifulSoup)
Software Development	GUI apps (Tkinter, PyQt), games (Pygame)
DevOps	Scripting for CI/CD pipelines
Cybersecurity	Pen-testing tools like Scapy, Nmap scripting

◇ PYTHON VERSIONS

- **Python 2.x**: Legacy, now deprecated.
- **Python 3.x**: Actively developed and widely used (latest as of 2025: 3.12+).

◇ WHO SHOULD LEARN PYTHON?

- ☑ Beginners in programming
- ☑ Data analysts/scientists
- ☑ Backend web developers
- ☑ Automation/test engineers
- ☑ AI/ML engineers

PYTHON LEARNING ROADMAP

STAGE 1: BASICS (BEGINNER)

- **Goal**: Understand syntax, data types, and control flow

Topics	Description
Python Setup & IDEs	Install Python, use VSCode, Jupyter, etc.
Variables & Data Types	Strings, Integers, Floats, Booleans
Input/Output	input(), print()
Operators	Arithmetic, comparison, logical
Conditional Statements	if, elif, else
Loops	for, while, break, continue
Functions	def, parameters, return values
String Manipulation	Slicing, formatting, string methods
Lists & Tuples	Indexing, slicing, methods
Dictionaries & Sets	Key-value access, operations

🎯 **Mini Projects**: Calculator | Number guessing game | Basic to-do list

STAGE 2: INTERMEDIATE PYTHON

- **Goal**: Learn data structures, file handling, and OOP.

Topics	Description
File Handling	open(), read/write text & CSV files
Exception Handling	try, except, finally, custom errors
List Comprehensions	One-liners for loops in lists
Lambda Functions	Anonymous short functions
Map, Filter, Reduce	Functional programming concepts
Modules & Packages	Creating and importing your own
Object-Oriented Programming	class, __init__, inheritance, encapsulation

☑ STAGE 3: ADVANCED PYTHON

- **Goal:** Learn powerful tools and libraries.

Topics	Description
Decorators	Wrap functions for added behavior
Generators & Iterators	Memory-efficient looping
Working with JSON, XML	APIs, data serialization
Regular Expressions	Text matching & parsing
DateTime Module	Handling dates and times
Multithreading & Multiprocessing	Parallel tasks
Virtual Environments	venv, pipenv for project isolation

☑ STAGE 4: LIBRARIES & REAL-WORLD USAGE

- **Goal:** Get hands-on with popular Python libraries.

Area	Libraries & Tools
Data Analysis	Pandas, NumPy
Visualization	Matplotlib, Seaborn, Plotly
Web Development	Flask, Django, FastAPI
Automation	Selenium, Requests, BeautifulSoup
APIs	requests, FastAPI, RESTful APIs
Machine Learning	Scikit-learn, TensorFlow, PyTorch
Testing	PyTest, Unittest

☑ STAGE 5: PROJECT DEVELOPMENT & DEPLOYMENT

🔹 **Goal**: Build and host real-world applications.

Topics	Description
Git & GitHub	Version control
Unit Testing	Writing & running tests
CI/CD Basics	GitHub Actions, Jenkins
Docker (Basics)	Containerize apps
Hosting Python Apps	Render, Heroku, AWS, Railway
Streamlit / Gradio	Build simple ML/data apps
Portfolio Building	Showcase your work on GitHub

🧭 **Tips to Stay on Track:**

- ✔ Practice daily (1–2 hours)
- ✔ Use **LeetCode**, **HackerRank** for problem solving
- ✔ Build projects for each major topic
- ✔ Join Python communities (Reddit, Discord, GitHub)
- ✔ Follow real-world documentation

PYTHON INTERVIEW QUESTIONS

BASIC QUESTIONS

Question	Answer	Tip to Answer
What are Python's key features?	Easy to learn, interpreted, dynamically typed, OOP support, cross-platform, large standard library	Emphasize simplicity, readability, and flexibility
Difference between list and tuple?	List: mutable, Tuple: immutable	Mention use cases: lists for dynamic data, tuples for fixed data
How is memory managed in Python?	Python uses a private heap and automatic garbage collection (GC) via reference counting	Mention gc module and memory management strategy
Difference between is and ==?	is: compares identity; ==: compares values	Use simple example: a is b vs a == b
Python data types?	int, float, str, bool, list, tuple, dict, set, etc.	Group into: Numeric, Sequence, Mapping, Set, Boolean, Binary

INTERMEDIATE QUESTIONS

Question	Answer	Tip to Answer
What are *args and **kwargs?	*args passes variable-length positional args, **kwargs passes keyword args	Use a short function example to explain
What is list comprehension?	A concise syntax for creating lists: [x for x in range(5)]	Emphasize better readability and speed
How to handle exceptions?	Use try, except, else, finally blocks	Mention use of finally for clean-up
Deep vs Shallow Copy?	Shallow: references same nested objects, Deep: completely new copy	Mention copy.copy() vs copy.deepcopy()
What is a lambda function?	An anonymous one-liner function: lambda x: x + 1	Use example and explain use in map, filter

ADVANCED QUESTIONS

Question	Answer	Tip to Answer
What is a decorator?	A function that wraps another function to modify its behavior	Use @decorator syntax in your explanation
What are generators?	Functions that yield items one by one using yield, saving memory	Mention lazy evaluation and memory efficiency
What is GIL in Python?	Global Interpreter Lock — ensures only one thread runs bytecode at a time	Mention its impact on CPU-bound multithreaded programs
Static vs Class method?	Static method: no access to class or instance. Class method: access via cls	Use code example to clarify decorators
How to handle large files?	Use generators, line-by-line reading, or pandas with chunksize	Emphasize memory usage optimization

TECHNICAL INTERVIEW QUESTIONS

Question	Answer	Tip to Answer
How is memory managed in Python?	Through private heap space, automatic garbage collection using reference counting and gc module	Mention Python's built-in garbage collector and memory optimization
What is the difference between is, ==, and id()?	is: identity, ==: value, id(): returns memory address	Use an example with mutable objects for clarity
What is the difference between Python 2 and Python 3?	Python 3 has better Unicode support, print() as function, improved syntax and libraries	Just mention Python 2 is outdated, and focus on Python 3 improvements
What is the difference between a shallow copy and a deep copy?	Shallow copy copies references to nested objects; deep copy clones everything recursively	Use copy.copy() and copy.deepcopy() in explanation
How do you handle exceptions in Python?	With try, except, else, and finally blocks	Mention you can catch specific exceptions for better error handling
How can you improve performance of a Python script?	Use generators, optimize loops, use built-in functions, avoid global variables, profile code using cProfile	Give real examples like replacing loops with comprehensions
What are Python modules and packages?	Module: single Python file. Package: folder containing __init__.py and multiple modules	Explain how importing works and its role in code reuse
What is the use of with statement?	It simplifies resource management like file handling, automatically closes file after use	Use example: with open('file.txt') as f:
What are comprehensions in Python?	A concise way to create sequences: list, dict, and set comprehensions	Use examples: [x for x in range(10)], {k: v for k, v in dict.items()}
How do you read/write a file in Python?	Use open(), read(), write(), and with statement for file handling	Mention file modes: 'r', 'w', 'a', 'rb', etc.
What is the difference between Python scripts and modules?	Script: file meant to be executed. Module: file meant to be imported	Add that a module can be reused across projects
What is the difference between del, remove(), pop(), and clear()?	del: deletes by index or entire object; remove(): removes by value; pop(): removes and returns; clear(): removes all	Explain use cases on lists and dictionaries

SCENARIO-BASED QUESTIONS

Scenario-Based Question	Answer Summary	Tip to Answer
You get a large CSV file with millions of rows. How would you load and process it efficiently?	Use pandas.read_csv() with chunksize, or use generator-based line reading	Mention memory optimization and avoiding full data load at once
You have a list of 10,000 records. How would you remove duplicates?	Use list(set(my_list)) for simple data or pandas.drop_duplicates() for structured data	Choose method based on data type and order preservation needs
A file operation crashes the script. How would you handle it gracefully?	Use try-except-finally and with open() to manage resources safely	Emphasize exception handling and resource cleanup
How would you debug a script that is giving inconsistent results?	Use print() statements, logging, pdb, and test in smaller chunks	Show structured debugging mindset
You need to check performance of a function. What do you do?	Use timeit, cProfile, or custom timer with time module	Mention analyzing bottlenecks and optimizing
How do you design a function that receives any number of inputs?	Use *args for positional and **kwargs for keyword arguments	Show flexibility and examples of usage
You're asked to build a reusable data validation utility. What would you do?	Write a function/module with parameterized rules and return validation messages or flags	Talk about modularity, reusability, and test cases
You're working on a team project. How do you make your Python code readable and standard?	Follow PEP8, add docstrings, use clear variable names, and structure into functions and modules	Mention linters (like flake8) and version control
A module you wrote is used in multiple scripts. How do you manage changes?	Refactor into a package, version it, and use virtual environments for isolation	Mention Git versioning, dependencies, and backward compatibility
You're scraping data and encounter rate-limiting. What's your approach?	Use time.sleep(), rotate user agents/IPs, or use APIs if available	Mention responsible scraping and best practices